

Checkpoint 3 - Mob10

Data de referencia: 26/05/2026

1. Visao geral

O projeto Mob10 evolui a base do checkpoint 2 para um sistema de console com foco em mobilidade urbana acessivel. Nesta etapa, o objetivo principal foi consolidar os requisitos de Programacao Orientada a Objetos, adicionar um menu interativo funcional, validar entradas do usuario e garantir operacoes CRUD nas listas mantidas pelos services.

2. Requisitos atendidos

- ? heranca implementada nas familias `Usuario` e `Veiculo`
- ? polimorfismo com metodos sobrescritos nas subclasses
- ? sobrecarga de construtores e metodos
- ? menu interativo funcional em `view/MenuAplicacao.java`
- ? validacao de entrada para campos sensiveis
- ? CRUD com `ArrayList` nos services
- ? comentarios no codigo explicando a logica central
- ? repositorio organizado no GitHub

3. Perguntas orientadoras

1. A heranca foi implementada corretamente? Onde?

Sim. A heranca foi implementada em dois pontos principais:

- ? `Usuario` -> `Motorista` e `Passageiro`
- ? `Veiculo` -> `CarroComum`, `CarroAdaptado` e `Van`

Isso permite reaproveitar atributos comuns, como dados de cadastro do usuario e dados basicos do veiculo, enquanto cada subtipo adiciona seu proprio comportamento.

2. O polimorfismo (override) esta presente? Em quais metodos?

Sim. O polimorfismo aparece principalmente nos seguintes metodos sobrescritos:

- ? `Motorista.exibirPerfil()`
- ? `Passageiro.exibirPerfil()`
- ? `CarroComum.exibirDetalhes()`

- ? ``CarroAdaptado.exibirDetalhes()``
- ? ``Van.exibirDetalhes()``
- ? ``CarroComum.calcularTarifa(double tarifaBase)``
- ? ``CarroAdaptado.calcularTarifa(double tarifaBase)``
- ? ``Van.calcularTarifa(double tarifaBase)``
- ? ``toString()`` nas classes de dominio

No menu, o metodo ``exibirPerfil()`` e chamado por meio da referencia ``Usuario``, demonstrando comportamento polimorfico em tempo de execucao.

3. Voce usou sobrecarga de metodos ou construtores?

Sim. A sobrecarga foi usada para flexibilizar a criacao dos objetos:

- ? ``Motorista(...)`` com e sem parametro de disponibilidade
- ? ``Passageiro(...)`` com e sem lista inicial de formas de pagamento
- ? ``Corrida(...)`` com construtor resumido para solicitacao padrao
- ? ``Corrida.calcularValor()`` e ``Corrida.calcularValor(double tarifaBase)``
- ? ``Pagamento(...)`` com e sem status informado
- ? ``Avaliacao(...)`` com e sem comentario
- ? ``Van(...)`` com e sem parametro explicito de adaptacao

4. O menu permite todas as operacoes necessarias?

Sim. O menu foi organizado por entidade e cobre o fluxo principal do sistema:

- ? passageiros: cadastrar, listar, atualizar e remover
- ? motoristas: cadastrar, listar, atualizar e remover
- ? corridas: solicitar, listar, atualizar rota, despachar, finalizar, cancelar e remover
- ? pagamentos: processar, listar, atualizar metodo e remover
- ? avaliacoes: registrar, listar, atualizar e remover

5. As validacoes impedem entradas incorretas?

Sim. O sistema valida:

- ? IDs positivos e unicos
- ? email em formato valido
- ? telefone com 10 ou 11 digitos
- ? CNH com 11 digitos

- ? placa em padrao Mercosul
- ? data valida no formato `dd/MM/aaaa`
- ? horario valido no formato `HH:mm`
- ? nota de avaliacao no intervalo de 1 a 5

Tambem existem validacoes de negocio, como:

- ? nao permitir pagamento de corrida nao finalizada
- ? nao permitir pagamento duplicado para a mesma corrida
- ? nao permitir avaliacao de corrida nao finalizada
- ? nao permitir avaliacao duplicada para a mesma corrida

6. O codigo esta comentado explicando a logica POO?

Sim. Foram inseridos comentarios objetivos em pontos centrais:

- ? protecao contra IDs duplicados no cadastro
- ? controle de consistencia das listas nos services
- ? uso do polimorfismo no menu ao listar usuarios

Os comentarios foram mantidos curtos para nao poluir o codigo, mas explicam a intencao das partes mais importantes da modelagem.

7. Todos os integrantes do grupo estao utilizando o GitHub? Se sim, qual a frequencia dos commits? Incluir o link do repo do GitHub.

O link do repositorio e:

- ? `https://github.com/elizeueuzebio/mob10-a3`

Pelo historico atual visivel no repositorio local em 26/05/2026, existem 2 commits publicados na branch principal e ambos aparecem no historico com autoria de `Elizeu`.

Nesse momento, nao ha evidencias locais de commits dos demais integrantes. Para fortalecer a avaliacao do checkpoint 3, o ideal e que cada integrante realize commits pequenos e frequentes, por exemplo:

- ? 1 commit ao concluir modelagem POO
- ? 1 commit ao concluir menu e validacoes
- ? 1 commit ao concluir CRUD e testes
- ? 1 commit ao concluir documentacao ou relatorio

Essa distribuicao ajuda a demonstrar participacao real e continuidade do trabalho.

8. Comente como esta o relatorio final de entrega. Indique ate onde esta feito e traga fragmentos do relatorio para demonstracao, se possivel.

O relatorio final ja pode ser considerado iniciado e estruturado ate a parte tecnica principal. Com a base atual, ja estao prontos ou parcialmente prontos:

- ? contextualizacao do projeto
- ? descricao da modelagem POO
- ? descricao das funcionalidades do sistema
- ? explicacao do menu, validacoes e CRUD
- ? respostas orientadoras do checkpoint 3

Os pontos que ainda podem ser aprofundados na versao final sao:

- ? diagrama de classes revisado
- ? capturas de tela ou fotos da execucao do menu
- ? conclusao final do grupo
- ? consideracoes sobre proximos passos do projeto

4. Fragmentos do relatorio final

Fragmento 1 - Introducao

"O projeto Mob10 foi desenvolvido com o objetivo de simular uma plataforma de mobilidade acessivel, conectando passageiros com necessidades especificas a motoristas com veiculos compatíveis. A proposta prioriza inclusao, autonomia e seguranca no deslocamento urbano."

Fragmento 2 - Modelagem POO

"A modelagem do sistema foi organizada com base em conceitos fundamentais de Programacao Orientada a Objetos. A classe `Usuario` concentra dados comuns de cadastro, enquanto `Motorista` e `Passageiro` especializam o comportamento conforme o papel de cada ator no sistema. Da mesma forma, a classe `Veiculo` foi especializada em `CarroComum`, `CarroAdaptado` e `Van`, permitindo reutilizacao de estrutura e uso de polimorfismo no calculo de tarifa e na compatibilidade com passageiros."

Fragmento 3 - Funcionalidade

"Na camada de apresentacao, foi criado um menu interativo via console que permite cadastrar, listar, atualizar e remover entidades do sistema. O fluxo principal contempla solicitacao de corrida, despacho de motorista, finalizacao, pagamento e avaliacao. O uso de validacoes reduz entradas invalidas e melhora a confiabilidade da execucao."

5. Critérios de avaliação

Modelagem POO

A estrutura possui classes, encapsulamento, herança, polimorfismo e listas de relacionamento.

Implementação POO

Os comportamentos foram distribuídos entre model, service e view, evitando concentrar toda a lógica na classe principal.

Funcionalidade

O sistema compila, executa e permite demonstrar o fluxo principal do aplicativo pelo menu.

Relatório

Já existe base textual suficiente para gerar o PDF do checkpoint 3, faltando apenas ajustes visuais, diagrama e evidências finais.

Apresentação

O projeto já possui menu funcional para demonstração ao vivo, o que facilita explicar o domínio, as regras de negócio e as validações implementadas.

GitHub

O repositório está público, com README e histórico inicial de commits. O próximo passo ideal é ampliar a frequência de commits do grupo durante a finalização do trabalho.